ClipForge AI Video Editor — Complete Architecture & Working Guide

A fully **in-browser, local-first AI video editor**. React 19 + TypeScript + Zustand/Immer/zundo + @tanstack/react-router, with WebCodecs, WebGPU (onnxruntime) and WASM doing all heavy processing client-side. Raw media never leaves the browser; AI calls go out only through user-configured provider keys (NVIDIA NIM, OpenRouter, ElevenLabs, etc.).

Table of Contents

---

1. Boot Sequence & Routing

`src/main.tsx`

The application boots in this order:

1. **Hydrate API config** — `useApiConfigStore` is hydrated from IndexedDB (encrypted local provider keys) before anything renders, so the UI never flashes "no provider".
2. **Apply theme** — `initTheme()` reads the persisted `clipforge-*` theme preference.
3. **Preload fonts** — `preloadEssentialFonts()` warms the fonts used by the text engine.
4. **Request persistent storage** — `navigator.storage.persist()` is requested so OPFS media (the largest data) is never evicted by the browser.
5. **Register the service worker** — `public/sw.js` in production (cache-first for built assets).
6. **Mount UI** — `<PassphraseGate><RouterProvider/>` . `PassphraseGate` lets the user unlock/encrypt their locally-stored API keys with a passphrase.

Router (`src/router.tsx`)

A hand-built (non-file-based) route tree using `@tanstack/react-router` :

Route	Page	Purpose
`/`	Landing	Marketing/hero + CTA
`/editor`	EditorPage	The full editor (bare full-viewport chrome)

`/settings`	Settings	Provider API keys, theme, preferences
`/studio` (legacy)	redirect	Redirects to `/editor`
`/app` (legacy)	redirect	Redirects to `/editor`

`src/router-components.tsx` wraps the editor in:

- `EditorErrorBoundary` — crash isolation.
- `BrowserGate` — capability check: WebGPU (with Canvas2D/WebGL2 fallback) — otherwise a friendly "browser not supported" screen.
- Suspense with `WebGPULoadingScreen`.

App shell (`src/ui/common/AppShell.tsx`)

- On `/editor`: renders the bare full-viewport (the editor owns its own chrome and layout).
- On landing/settings: header + mobile bottom nav.

---

2. Data Model

All types live in `src/engine/types.ts`.

Project

- `width` × `height`, `fps`, `aspectRatio`
- 8 default tracks (2× video, 2× audio, 2× text, 2× fx)
- `markers: ClipMarker[]`
- `captions: CaptionsConfig`
- `schemaVersion` — old projects are upgraded via `migrateProjectTracks()`.

Track

Four types: `video | audio | text | fx`. Flags: `lock`, `mute`, `hidden`, `solo`. Tracks carry *subtype badges* (styling hints):

Track type	Subtypes
video	`video`, `image`, `avatar`, `animation`, `slide`
audio	`audio`, `music`, `voice`, `sfx`
text	`caption`, `title`, `lowerThird`, `sticker`, `callout`
fx	(none)

Clip — the universal editable unit

- **Timing**: `startTime`, `duration`, `sourceStart`, `sourceEnd`, `speed`
- **Transform**: `position`, `scale`, `rotation`, `opacity`, `anchor`
- **Audio**: `volume`, `fadeIn`, `fadeOut`, `eq` (3-band low/mid/high), `duckUnderTrackId`, `preservePitch`
- **Visuals**: `effects[]`, `transitions`, `blendMode`, `crop`, `border`, `dropShadow`
- **Motion**: `keyframes[]`
- **Text**: optional `text: TextOverlay` (font, size, color, alignment, animation)

- **Metadata**: `clipType`, `fitMode` (`cover | contain`), `avatarRole`, `createdBy: 'director' | manual` (drives grouped undo for AI work)

EffectType (14)

`brightness, contrast, saturation, vibrance, temperature, tint, hue, blur, grayscale, vignette, grain, chromatic-aberration, glitch, morph`.

> *Note: `morph` is declared in the type union but is **not implemented** in the compositor.*

---

3. Storage — a Three-Tier Split

Tier	Contents	Where
OPFS	Raw media bytes under `clipforge-media/<assetId>/<file>`	`src/engine/storage/opfs.ts` (path-traversal validated)
IndexedDB (`clipforge-app` v3)	`projects`, `assets` (metadata + `filePath`), `settings` (cached per-asset analysis: `transcript:<id>`, `scenes:<id>`, `ocr:<id>`), `history`	`src/engine/storage/db.ts`
localStorage	UI preferences via `clipforge-*` keys (panel widths, open states, ai mode, guides) + encrypted API config	stores + `api/config`

Key rule: **assets reference media by OPFS-relative path**. `getMediaUrl(path)` converts that to a blob URL on demand — the DOM never holds the path.

---

4. State Management & Undo/Redo

`src/stores/timelineStore.ts`

The document lives in **one Zustand store**, wrapped with:

- **zundo** — temporal snapshots for undo/redo
- **immer** — structural sharing / copy-on-write

History instrumentation

Snapshots are **paused**; recording is *explicit and manual*:

1. Every mutation goes through `mutate(action)`.
2. `beginHistory(meta)` snapshots the pre-state `project` reference + a `UiCheckpoint` (selection + playhead).
3. Split/clip edits called `mutate()` individually; composite user gestures (a drag session, a whole AI plan) call `commitHistory()` once.
4. `pastStates` cap = 50. Redo stack cleared whenever a new snapshot commits.

History groups & transactions

- `beginHistoryGroup()/endHistoryGroup()` — collapse **multi-step ops** (an entire drag, an entire trim) into a **single undo step**.
- `withTransaction(fn)` — async batches (e.g. import pipeline) that undo as one step.

- `suspendHistory(true)` — used by the AI Director so an entire autonomous plan becomes one undo step.

`undo()/redo()`

Restore the snapshot, step the human-readable log (`historyStore`), fire a toast, and restore the UI checkpoint (selection + playhead). The **log + snapshots persist across reloads**.

Autosave

Every mutation:

1. Sets `dirty = true` .
2. Schedules a **2s-debounced** IndexedDB save (`autosave()`).
3. `flushSave()` is hooked to `beforeunload` / `visibilitychange` .

Hydration

On load: load newest project from IDB, or build the **Welcome Project** on first run; seed history from persisted snapshots.

---

5. Rendering & Playback Engine

One compositor, three consumers

`compositeFrame(ctx, project, assets, time, media)` at **`src/engine/render/composite.ts:270`** is the *single source of truth for a frame*. It is shared by:

1. The live preview loop (`usePlayback.ts` → `paint()`).
2. The **WebM** export pipeline.
3. The **MP4** export pipeline.

This guarantees **preview == exported output**.

Playback loop (`src/hooks/usePlayback.ts:368`)

- One `requestAnimationFrame` loop advances a wall-clock base: `time = clock.base + elapsed * speed` .
- Supports **negative speed (J-shuttle)**.
- Pauses during export via the ref-counted `isExportActive()` .
- Only repaints when time moved `> 0.001s` **or** a `repaintToken` was set (store subscription / media events).

Media pooling

- `<video>` / `<audio>` elements live in a hidden 2px host div (Chrome will not present frames otherwise).
- Cached **per-asset** — never recreated on seek.
- **1× free-run**: while playing at 1×, elements free-run (they are the clock); Drift only resynced at `> 0.25s` — setting `currentTime` every frame freezes Chrome, so this heuristic avoids it.
- Otherwise elements are explicitly seeked frame-by-frame.

Compositing order per frame (`composite.ts:270-597`)

1. Clear canvas.
2. Gather active video + text clips (Z-order = reversed track index).
3. Per clip:

    - Keyframe-interpolated transform/opacity.
    - **Transition alpha — pure dissolve regardless of** `type` (wipe/slide/zoom are not geometrically rendered).
    - CSS-effect filter string from `effects[]`.
    - Blend mode, drop shadow.
    - Video draw with **smart reframing/auto-crop** (`interpolateCrop`) or manual crop, `fitMode` sizing, borders.
    - Chromatic-aberration + glitch re-draw passes.
    - Image draw.
4. Grain overlay — cached 128px noise tile.
5. **Text overlays** — stacked; 10 animation types (eased); Google Fonts loading; stroke/shadow/underline/typewriter.
6. **Auto-caption layer** (`makeCaptionsProvider`) — sentence or **karaoke word-by-word**; avoids OCR-detected protected regions.
7. Vignette radial gradient.

WebGPU reality

The timeline framebuffer is **Canvas2D**, not WebGPU. WebGPU runs only where it matters — ONNX Runtime (onnxruntime `webgpu` EP) inference inside purpose-built workers:

- **MODNet background removal**
- **Wav2Lip**
- **Whisper (ASR)**

Motion graphics render via WebGL2/WebGPU inside a **sandboxed iframe** (see §11) and export to WebM via WebCodecs.

---

6. Timeline Interactions

`src/ui/timeline/Timeline.tsx` is a huge presentational component driven directly by `timelineStore` (document state) and `editorStore` (UI state). Full interaction surface:

- **Drag-move** with magnetic snapping (playhead / clip edges / markers; Shift inverts snap).
- **Trim handles** — adjust `sourceStart`, `sourceEnd`, `duration`.
- **Marquee** multi-select.
- **Ruler drag-scrub**.
- **Zoom** — slider, Ctrl ±, fit.
- **Split / cut / duplicate / delete / ripple** via toolbar.
- **Right-click context menu** — split, duplicate, cut, ripple-delete, speed cycle, jump-to-start/end, hard delete.
- **Floating multi-clip action bar** when >1 clip selected.
- **Hover audio-clip bar** — denoise / split / trim / delete.
- **Tools** — scissors (razor), rate, text.
- **Keyboard** — `M` markers, JKL shuttle, arrow nudge, `[` / `]` edge-trim, Space play/pause.

Undo hygiene: every drag/gizmo gesture = **one undo step** via history groups. `trackRectsRef` caches track bounds to avoid per-frame DOM reads.

---

7. Audio Pipeline

Export mixing (`src/engine/export/audioMix.ts`)

An `OfflineAudioContext` builds the graph per clip:

```
BufferSourceNode (speed, preservePitch)
  → optional 3-band EQ (BiquadFilter low/mid/high)
  → GainNode (fade-in/fade-out ramps)
  → optional ducking GainNode (dips to 20% under triggering tracks)
  → destination
```

Solo / mute / volume / master are applied; then the graph renders offline to an `AudioBuffer` for muxing.

Editing playback (`usePlayback.syncAudio`)

No Web Audio graph at edit time (no latency). Per frame it sets:

```
el.volume = clipVolume × mute × solo × master × duckFactor
```

and plays/pauses pooled `<audio>` elements.

TTS (`src/api/tts/`)

- Providers: **Magpie** (NVIDIA NIM), **ElevenLabs** — behind a `TtsProvider` interface; `getActiveTtsProvider()` picks by user preference.
- Flow (used both by the Voiceover studio UI and the AI `generate_voiceover` tool): synthesize → blob → `importFiles()` → clip placed on an audio track.
- **Autonomous mode measures real audio duration** and places narration precisely on the timeline.

Denoise

RNNoise WASM (`@shiguredo/rnnoise-wasm`): resample to 48 kHz, process 480-sample frames, write via `float32ToWav` , then reimport as a new clip alongside the original.

Transcripts & captions

- Whisper (`@xenova/transformers`) runs **in a worker** (`captions-worker.ts`).
- Result = `StoredTranscript` (words + sentences), persisted per asset (`transcript:<id>`).
- Caption cues render over the canvas.
- Two caption systems:
  1. **Auto overlay** driven by `CaptionsConfig` (karaoke/sentence).
  2. **Manual caption text-clips** via the Captions editor (SRT/VTT export).

Music & SFX

- Music search: fan-out across **Deezer / MusicBrainz / Internet Archive**; MusicBrainz requests carry a UA header (required by their API).
- **Sound effects are procedurally synthesized in-browser** — no API key needed.
- Giphy stickers are GIF→WebM converted with WebCodecs and looped.
- Autonomous production auto-**ducks music by style**(energetic 0.25 ... cinematic 0.12).

---

8. The AI System

AI Director (`src/ui/ai/AIDirector.tsx`)

Floating, draggable, resizable, **position+size persisted**:

- `clipforge_ai_director_pos`
- `clipforge_ai_director_size`
- `clipforge-ai-director-launcher-pos`
- `ai_director_mode`

Three visual states: launcher orb → minimized pill → full panel.

Two production modes:

Mode	Behavior
Autopilot	Tools auto-apply.
Review	Staged proposal cards; `shouldAutoApply(toolName)` gates by `confirmationLevel` — *destructive* tools need confirm, *expensive* tools need confirm → Accept/Reject per card. All applied inside **one `withTransaction` undo step**.

Send loop (`send()`):

1. Detect "make a video" prompts → **6-step Video Brief wizard**.
2. Otherwise loop up to **6 chat turns** with the 42-tool registry:
    - execute tools
    - stage proposals
    - ask questions (`ask_user` , dedup + cancel-safe empty-answer resolution)
    - review quality
    - suggest follow-ups

Request pipeline (`src/api/llm/director.ts`)

- Builds an OpenAI-compatible chat request with an enormous system prompt (project state + full `VIDEO_EDITING_MANUAL` + a "CRITICAL EXECUTION MANDATE" that forces tool calls).
- Injects `DIRECTOR_TOOLS` , `tool_choice: 'auto'` .
- No streaming; 3 retries with backoff on transient errors.
- CORS-proxied through a validated allowlist — `src/lib/proxyHosts.ts` (13 hosts) — with AbortSignal propagation.

Tools registry (`src/api/llm/tools.ts` — 42 tools)

The LLM edits the timeline by calling tools. Categories:

Category	Tools
Project	`set_project_ratio`
Clip editing (destructive set)	`split_clip`, `trim_clip`, `move_clip`, `delete_clip`, `join_clips`
Properties	`set_clip_property`, `set_transition`, `add_text_overlay`
Assets	`search_stock_image`, `search_music`
Analysis	`understand_video`, `check_quality`, `review_project` (read-only)
Script	`generate_script`, `rewrite_script`, `shorten_script`, `expand_script`, `script_hook`, `script_cta`

Voice	`generate_voiceover` (TTS), `generate_captions` (Whisper)
Generative media	`generate_motion_graphics` (LLM writes canvas code), `generate_slides` (LLM writes Marp), `generate_avatar_*` (Wav2Lip)
Vision/ML	`smart_reframe`, `remove_background` (ONNX MODNet)
Output	`render_preview` (direct WebM export + auto-download)
Coordination	`ask_user`, `plan_edit` (multi-step proposal), `execute_autonomous_video_plan`, `dispatch_subagent_task`

Tools reference `destructive` / `expensive` flags that drive `shouldAutoApply`.

Subagents (`src/ai/subagents/`)

Roles (7): `script_architect`, `audio_producer`, `visual_animator`, `asset_curator`, `timeline_editor`, `motion_subtitler`, `quality_critic`.

Orchestrator (`SubagentOrchestrator.ts`) builds a fixed plan:

```
aspect → research → script → scene sequence → music → quality → render
```

It runs the plan **sequentially**, in **one undo step**, with:

- **Provider preflight** — blocks if LLM/TTS not configured.
- Event-emitted progress.
- A completion gate.

`scriptToPlan.ts` turns the stored script into timed scenes:

1. TTS narration clip.
2. Auto-fitted visuals: stock image → user media → deterministically-rendered SVG card (fallback).
3. Caption overlay.

Everything tagged `createdBy: 'director'` → single grouped undo.

Intent detection

- `ContextUnderstandingEngine` : keyword classifier → video type / audience / tone / duration / visual strategy.
- `videoBrief.ts` : `isVideoCreationPrompt` regex gate routes to the brief wizard.
- The LLM itself is the tool-choice driver.

---

9. Export Pipeline

Entry: `ExportDialog` → `exportVideo` (WebM) or `exportMp4` (MP4). Both share a common core.

Shared core

- `beginExportSession()` (`src/engine/export/exportSession.ts:20`) — ref-counted; pauses preview while an export runs.
- Offscreen canvas.
- Per-frame: `compositeFrame(ctx, …)` → `VideoFrame` → `VideoEncoder`.
- **Backpressure**: `waitForDrain(encoder, queueLimit)`.

- `EncoderGuard` — turns encode errors into rejections (fail loud, not hang).
- Keyframe every 2 s; 30-frame media eviction.
- Throttled progress (~120 ms) + AbortController cancel.

WebM (`src/engine/export/exportVideo.ts`)

- Hand-rolled **EBML muxer** (`webm-muxer.ts`) — byte-level vints, clusters per keyframe, cue points. No dependency.
- Codecs: vp8 / vp9 / av1.
- Audio: `OfflineAudioContext` mix → Opus encode.

MP4 (`src/engine/export/exportMp4.ts`)

- WebCodecs **H.264** (`avc1.*` fallback chain).
- **AAC** audio, with an `AudioEncoder.isConfigSupported` probe so a malformed empty audio track is never produced.
- Muxed via `mediabunny` into a Blob.

Bitrate & formats

- `bitrateFor(quality, width, height)` (`src/lib/exportFormats.ts:71`) — resolution-scaled bitrate, floor 800 kbps.
- Live size estimate dialog.
- Requires WebCodecs — the `frames` format is a documented placeholder only.
- The AI can trigger export directly via `render_preview` .

---

10. UI Surfaces: Studios, Panels & Modals

RightToolPanel — 16 sections

`text, insights, effects, audio, voiceover, captions, transitions, stickers, speed, keyframe, crop, slides, avatar, design, script, images`

Rendered as a desktop drawer (slides over the canvas) or a **mobile bottom sheet**, driven by `editorStore.toolPanelSection` .

Studios

Studio	Purpose
`ScriptStudioModal`	Script writing + teleprompter + camera recording
`SlideStudioModal`	Marp decks → PNG slides
`CaptionsEditor`	Whisper ASR + manual caption clips; SRT/VTT export
`LipSyncEditor`	Wav2Lip + avatar face presets

Inspector (`src/components/inspector/`)

Transform controls with **batched history** (500 ms collapse), keyframe diamonds, effects, blend modes, EQ/duckingsettings, normalize/denoise/trim-silence, transitions, and multi-clip batch editing.

Media Bin

- Tabs: **Project Media / AI Assets / Stock Search**.

- Search, filter, sort, grid.
- Import pipeline: OPFS write → probe → thumbnail → proxy/filmstrip/waveform → IDB metadata.
- Webcam + screen recording (MediaRecorder).
- Stock search: Unsplash / Pexels / Giphy.
- Clipboard-paste import.
- Module-scope HTML5 drag state (DnD events).

Global chrome

- **Command palette** (Ctrl+Shift+P) — studio/panel/project/edit actions, keyboard nav (`src/components/editor/CommandPalette.tsx`).
- Shortcuts cheat-sheet (`?`).
- History panel + toasts.
- Export / New / Open dialogs — opened via **window events** (`src/lib/uiEvents.ts`).
- NewProject presets.
- Onboarding tour.
- PassphraseGate.
- Capability banner (missing WebCodecs/WebGPU notice).
- Error boundaries.

---

11. Workers & Wasm Inference

```
Worker tree
├─ engine/workers/          (decode / render / encode / ai) — health-check stubs
├─ motionPreview.worker.ts  (runs user/AI canvas code via OffscreenCanvas ping-pong)
├─ mediaProcessor.ts        (thumbnails)
├─ captions-worker.ts       (Whisper ASR via @xenova/transformers)
├─ denoise-worker.ts        (RNNoise WASM)
└─ lipsync-worker.ts        (Wav2Lip — onnxruntime-web)
```

Motion graphics sandbox (`engine/motion/sandbox.ts`): user- or LLM-generated canvas code executes inside a **sandboxed iframe** (WebGL2/WebGPU), rendered to an OffscreenCanvas ping-pong, and exported to WebM via WebCodecs — so untrusted generative code never touches the main timeline context.

---

12. Key Architectural Patterns

1. **One compositor, three consumers** — preview and both exports call `compositeFrame()` ; frame-for-frame identical output.
2. **Explicit undo instrumentation** — history is recorded, grouppable, interruptible, and persisted.
3. **Element pooling + 1× free-run** — the DOM-video trick that keeps playback smooth and feature-complete.
4. **Manual EBML muxing** — MP4/WebM without a server or native plugin.
5. **The LLM as editor operator** — a 42-tool API (with a human equivalent) is its interface to the timeline.
6. **Sequential autonomous "production line"** — provider preflight, single-step undo, progress events, completion gate.
7. **Local-only processing** — WebGPU/WASM ML, CORS-proxied AI calls, encrypted local keys.

---

13. File Map

Area	Files

Boot / routing	src/main.tsx, src/router.tsx, src/router-components.tsx, src/ui/common/AppShell.tsx, src/ui/common/PassphraseGate.tsx
Data model	src/engine/types.ts
Storage	src/engine/storage/{db,opfs}.ts, src/api/config/store.ts
State	src/stores/{timelineStore,editorStore,aiStore}.ts, src/stores/historyStore.ts
Rendering / playback	src/engine/render/composite.ts, src/hooks/usePlayback.ts, src/components/editor/PreviewCanvas.tsx, src/ui/preview/Preview.tsx
Export	src/engine/export/{exportVideo,exportMp4,exportSession,audioMix,webm-muxer}.ts, src/lib/exportFormats.ts, src/ui/export/ExportDialog.tsx
Audio	src/engine/export/audioMix.ts, src/api/tts/*, src/workers/{captions,denoise,lipsync}-worker.ts
AI	src/ui/ai/AIDirector.tsx, src/api/llm/{director,tools,askedQuestions,slides,scripts,plan,context}.ts, src/ai/subagents/*, src/api/proxy.ts, src/lib/proxyHosts.ts
Timeline	src/ui/timeline/Timeline.tsx, src/components/timeline/*, src/hooks/useTimeline*
UI panels	src/ui/common/RightToolPanel.tsx, src/ui/script/ScriptStudioModal.tsx, src/ui/slides/SlideStudioModal.tsx, src/components/media/*, src/components/inspector/*, src/components/editor/CommandPalette.tsx, src/lib/uiEvents.ts
Media bin	src/components/media/*, src/workers/mediaProcessor.ts
Motion / ML	src/engine/motion/sandbox.ts, src/ai/motion/*, src/ai/video/*, src/workers/motionPreview.worker.ts